

```

#pragma once
#include <string>
#include <utility>
#include <vector>
#include <iterator>
#include <string.h>

/*
Part 1: Build Your Own Linked List
Implement and test a linked list that supports adding, searching, removing, and printing items. Do
not use std::list.
**Made adding the new nodes
**Printed new nodes
**Made the remove node

Part 2: Monopoly Game
Use your linked list to build a circular board with at least 10 properties.
**Made my linked list to a doubly linked list
**Made atleast 10 properties

Each property stores its name, cost, owner, and next pointer.
**Made all of them

Players move around the board, wrapping back to the beginning.
**Made it loop around

Players can buy unowned properties, but cannot buy owned properties.
**Can buy empty properties but not owned

Simulate at least 10 turns and print the results

BONUS POINTS IF YOU CREATE A GUI 😊
**Created a GUI assisted with QT
Briefly explain the time complexity of adding, searching, removing, moving, and traversing.

*/

//*****
//*****
//%%%%%%%%%% WE ARE GOING TO MAKE THIS WHOLE FILE OUR BOARD THAT INCLUDES PROPERTY
//%%%%%%%%%%
//*****
//*****

//Creates my linked list with a default head null pointer if empty
struct Property{
    std::string name;
    int cost;
    int owner = -1;
    Property* next = nullptr;
    Property* prev = nullptr;

    Property(std::string name, int cost) : name(std::move(name)), cost(cost){}
};

class Board {
private:
    Property* head = nullptr;
    Property* tail = nullptr;
    int count = 0;

```

```

public:
    Board() = default;
    Board(const Board&) = delete;
    Board& operator = (const Board&) = delete;
    ~Board() {clear();}

    //Make a set function that adds a new node
    void append(const std::string& name, int cost){
        //Adds a new node to the heap
        Property* node = new Property(name, cost);

        //If empty make a the new node as head
        if(head == nullptr){
            head = tail = node;
            node->next = node;
            node->prev = node;
        }
        //Else if not empty, make the new node be sent to the tail
        else{
            tail->next = node;
            node->prev = tail;
            node->next = head;
            head->prev = node;
            tail = node;
        }
        ++count;
    }
    //Use this to remove values, will search along the array to do so
    bool remove(const std::string& name){
        Property* node = search(name);
        if(!node){
            return false;
        }

        if(count == 1){
            head = tail = nullptr;
        }

        else{
            node->prev->next = node->next;
            //Assigns previous to point to the previous node
            node->next->prev = node->prev;

            //If our temp value is head point the new head next
            if(node == head){
                head = node->next;
            }
            //If our temp value is tail, point to the previous node.
            if(node == tail){
                tail = node->prev;
            }
        }
        delete node;
        --count;
        return true;
    }

    //Use this to search for a value and print it
    Property* search(const std::string& name) const {
        if(!head){
            return nullptr;
        }

        Property* current = head;
    }

```

```

    do{
        if(current->name == name){
            return current;
        }
        current = current->next;
    }while(current != head);
    return nullptr;
}

//Using this to move arround the board
Property* move(Property* from, int steps, bool* passedStart = nullptr) const{
    if(passedStart){
        *passedStart = false;
    }
    Property* current = from;

    for(int i = 0; i < steps; i++){
        current = current->next;
        if(current == head && passedStart){
            *passedStart = true;
        }
    }
    return current;
}

//Move to Certain space
std::vector<Property*> toVector() const{
    std::vector<Property*> out;
    if(!head){
        return out;
    }

    Property* current = head;

    do{
        out.push_back(current);
        current = current->next;
    }while(current != head);

    return out;
}

Property* start() const {
    return head;
}

int size() const{
    return count;
}

void clear(){
    if(!head) return;
    tail->next = nullptr;
    Property* current = head;

    while(current){
        Property* next = current->next;
        delete current;
        current = next;
    }
    head = tail = nullptr;
    count = 0;
}

};

```

```
cmake_minimum_required(VERSION 3.16)
project(LinkedListMonopoly LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)

add_executable(Monopoly
    main.cpp
    mainwindow.cpp
    mainwindow.h
    board.h
    game.h
)

target_link_libraries(Monopoly PRIVATE Qt6::Widgets)
```

```

#pragma once
#include <random>
#include <string>
#include <vector>
#include "board.h"

using namespace std;

//Information on player/ player node
struct Player{
    string name;
    int money = 1300;
    Property* position = nullptr;
};

class Game{
//Setting up players and roll rng
private:
    Board board_;
    vector<Player> players_;
    int current_ = 0;
    bool rolled_ = false;
    mt19937 rng_{std::random_device{}()};

public:
    Game(){
        //Square places 19
        const char* names[] = {"Go", "Wall St", "Fleet St", "Broadway", "Champs-Elysees",
                                "Oxford St", "Abbey Rd", "Hollywood Blvd", "Fifth Ave", "Ginza St",
                                "Orchard Rd",
                                "Tocayo Ave", "Nanjing Rd", "Las Ramblas", "Nevsky Prospekt", "Via
                                Appia",
                                "Beale St", "Khasoan Rd", "Akihabara"};
        //Cost associated to square places
        const int costs[] = {0, 40, 100, 125, 150, 200, 210, 250, 300, 325, 345,
                              375, 390, 400, 410, 490, 520, 550};

        //Appends both names and cost to the board
        for(int i = 0; i < 16; i++){
            board_.append(names[i], costs[i]);
        }

        //Set the players at start
        for (const char* n : {"Player 1", "Player 2"}){
            Player p;
            p.name = n;
            p.position = board_.start();
            players_.push_back(p);
        }
    }
    //This is just setting up standard functions so that I can call further down below
    const Board& board() const{
        return board_;
    }

    const vector<Player>& players() const{
        return players_;
    }

    int currentIndex() const{
        return current_;
    }

    const Player& currentPlayer() const{

```

```

        return players_[current_];
    }

    bool hasRolled() const{
        return rolled_;
    }
    //////////////////////////////////////

    //Rolling the dice
    string roll(){
        uniform_int_distribution<int> die(1,6);
        int d1 = die(rng_);
        int d2 = die(rng_);

        Player& p = players_[current_];

        bool passed = false;
        p.position = board_.move(p.position, d1 + d2, &passed);
        rolled_ = true;

        string log = p.name + " rolled " + to_string(d1) + " + " + to_string(d2) +
            " and landed on " + p.position->name + ".";

        if (passed){
            p.money += 100;
            log += "\n Passed Go: +$100.";
        }

        Property* square = p.position;

        if(square->cost > 0 && square-> owner != -1 && square->owner != current_){
            int rent = square->cost / 2;
            p.money -= rent;
            players_[square->owner].money += rent;

            log += "\n Owner by " + players_[square->owner].name + ". Paid rent $" + to_string(rent)
+ ".";
        }
        else if(square->owner == current_){
            log += "\n You already own this.";
        }
        return log;
    }

    //Sets up the backend logic for buying sqaures
    bool canBuy() const {
        const Player& p = players_[current_];
        return rolled_ && p.position->cost > 0 && p.position->owner == -1 && p.money >= p.position-
>cost;
    }
    //Sets up the button to buy land (uses canBuy() to preverify buy() option)
    string buy(){
        if(!canBuy()){
            return "Can't buy this property.";
        }
        Player& p = players_[current_];
        p.money -= p.position->cost;
        p.position->owner = current_;
        return p.name + " bought " + p.position->name + " for $" + to_string(p.position->cost) + ".";
    }
    //Sets up the button logic to end turn
    void endTurn(){
        rolled_ = false;
        current_ = (current_ + 1) % static_cast<int>(players_.size());
    }

```

```
};
```

```
#include <QApplication>
#include "mainwindow.h"

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    MainWindow w;
    w.show();

    return a.exec();
}
```

```
#pragma once
#include <QMainWindow>
#include <QVector>
#include "game.h"

class QLabel;
class QPushButton;
class QTextEdit;

class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = nullptr);

private slots:
    void onRoll();
    void onBuy();
    void onEndTurn();

private:
    void buildUi();
    void refresh();

    Game game_;
    QVector<QLabel*> tiles_;
    QLabel* status_ = nullptr;
    QPushButton* rollBtn_ = nullptr;
    QPushButton* buyBtn_ = nullptr;
    QPushButton* endBtn_ = nullptr;
    QTextEdit* log_ = nullptr;
};
```

```

#include "mainwindow.h"
#include <QGridLayout>
#include <QLabel>
#include <QPushButton>
#include <QTextEdit>
#include <QVBoxLayout>
#include <QWidget>

namespace {
const int COLS = 5;
const int ROWS = 5;
const char* kPlayerColors[] = {"#ffb3b3", "#b3d1ff"};

//Helps out use qt to make the UI layout for board set up
QPair<int, int> ringPosition(int i)
{
    int idx = 0;
    for(int c = 0; c < COLS; ++c){
        if(idx++ == i){
            return {0, c};
        }
    }
    for(int r = 1; r < ROWS; ++r){
        if(idx++ == i){
            return {r, COLS - 1};
        }
    }
    for(int c = COLS - 2; c >= 0; --c){
        if(idx++ == i){
            return {ROWS - 1, c};
        }
    }
    for(int r = ROWS - 2; r >= 1; --r){
        if(idx++ == i){
            return {r, 0};
        }
    }
    return {0,0};
}
}

//Refreshes board to when you first start
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent){
    setWindowTitle("Linked List Monopoly");
    buildUi();
    refresh();
    log_>append("Game started. " + QString::fromStdString(game_.currentPlayer().name) + " goes first.");
}

//Where the buttons get mapped to, as well as the locations of the square plots player can buy
void MainWindow::buildUi(){
    auto* central = new QWidget(this);
    auto* grid = new QGridLayout(central);
    grid->setSpacing(4);

    //One QLabel per node in the linked list
    const auto nodes = game_.board().toVector();
    for (int i = 0; i < static_cast<int>(nodes.size()); ++i) {
        auto* tile = new QLabel(this);
        tile->setAlignment(Qt::AlignCenter);
        tile->setWordWrap(true);
        tile->setMinimumSize(120, 90);
        tile->setFrameStyle(QFrame::Box);
    }
}

```

```

        auto pos = ringPosition(i);
        grid->addWidget(tile, pos.first, pos.second);
        tiles_.push_back(tile);
    }

    // Center panel: status, buttons, log
    auto* center = new QWidget(this);
    auto* v = new QVBoxLayout(center);
    status_ = new QLabel(this);
    status_->setAlignment(Qt::AlignCenter);
    rollBtn_ = new QPushButton("Roll dice and move", this);
    buyBtn_ = new QPushButton("Buy property", this);
    endBtn_ = new QPushButton("End turn", this);
    log_ = new QTextEdit(this);
    log_->setReadOnly(true);

    v->addWidget(status_);
    v->addWidget(rollBtn_);
    v->addWidget(buyBtn_);
    v->addWidget(endBtn_);
    v->addWidget(log_);
    grid->addWidget(center, 1, 1, ROWS - 2, COLS - 2);

    setCentralWidget(central);

    connect(rollBtn_, &QPushButton::clicked, this, &MainWindow::onRoll);
    connect(buyBtn_, &QPushButton::clicked, this, &MainWindow::onBuy);
    connect(endBtn_, &QPushButton::clicked, this, &MainWindow::onEndTurn);
}

void MainWindow::refresh()
{
    const auto nodes = game_.board().toVector();
    const auto& players = game_.players();

    for (int i = 0; i < nodes.size(); ++i) {
        const Property* p = nodes[i];

        QString text = QString::fromStdString(p->name);
        if (p->cost > 0) text += QString("\n$%1").arg(p->cost);
        if (p->owner != -1)
            text += "\nOwner: " + QString::fromStdString(players[p->owner].name);

        // Show a token for every player standing on this node
        QString tokens;
        for (int k = 0; k < static_cast<int>(players.size()); ++k)
            if (players[k].position == p) tokens += QString(" [P%1]").arg(k + 1);
        if (!tokens.isEmpty()) text += "\n" + tokens;

        QString bg = (p->owner == -1) ? "#f4f4f4" : kPlayerColors[p->owner % 2];
        tiles_[i]->setText(text);
        tiles_[i]->setStyleSheet(QString("background:%1; color:black;").arg(bg));
    }

    QString s;
    for (int k = 0; k < static_cast<int>(players.size()); ++k)
        s += QString("P%1: $%2 ").arg(k + 1).arg(players[k].money);
    s += QString("\nCurrent: %1").arg(QString::fromStdString(game_.currentPlayer().name));
    status_->setText(s);

    rollBtn_->setEnabled(!game_.hasRolled());
    buyBtn_->setEnabled(game_.canBuy());
    endBtn_->setEnabled(game_.hasRolled());
}

```

```
void MainWindow::onRoll()
{
    log_>append(QString::fromStdString(game_.roll()));
    refresh();
}

void MainWindow::onBuy()
{
    log_>append(QString::fromStdString(game_.buy()));
    refresh();
}

void MainWindow::onEndTurn()
{
    game_.endTurn();
    refresh();
}
```

```
cmake_minimum_required(VERSION 3.16)
project(LinkedListMonopoly LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)

add_executable(Monopoly
    main.cpp
    mainwindow.cpp
    mainwindow.h
    board.h
    game.h
)

target_link_libraries(Monopoly PRIVATE Qt6::Widgets)
```